

Control flow

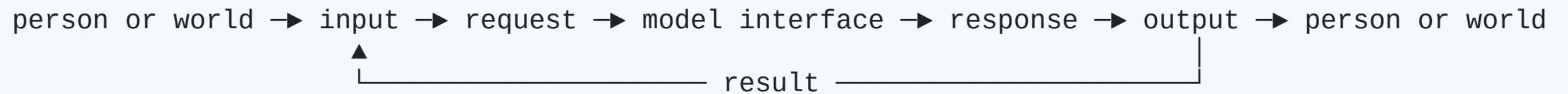
A hands-on course in building agents by building their harness

Lesson 3

Lesson 2 runs once, then stops

- Input, call, output, stop
- The model can ask for a tool, and the tool runs
- But the result never goes back to the model
- Something has to decide what happens next

Control flow is the path between the other four



- Input gathers, the model interface sends, output handles
- A tool's result is input again
- Control flow decides what happens at the end of the path

How you arrange them decides what you've built

- **Run once.** Input, call, output, stop. That's Lesson 2.
- **A chat loop.** Call, show the response, hand back to the person, repeat.
- **A workflow.** Your code decides the steps: draft, check, fix.
- **An agent loop.** While the model asks for tools, run them and call again.

The difference is who decides what happens next

- **Workflow:** your code decides
 - predictable and easy to test
 - only handles what you planned for
- **Agent:** the model decides
 - handles what you didn't plan for, and costs you predictability

Control flow sequences and terminates

- **Sequences:** passes what one primitive produced to the next
- **Terminates:** decides when to stop, or when to hand back to a person

A loop with no clear way to end is a bill with no clear way to end.

quark.py: Lesson 2, plus one loop

```
chat = len(sys.argv) < 2

# — control flow —
messages = [{"role": "user", "content": input}]

while True:
    output = call(max_tokens=16384, tools=tools, messages=messages).content
    messages.append({"role": "assistant", "content": output})
```

- `chat` is true when nothing came in on the command line
- Each pass of `while True:` is one call to the model
- `messages` carries everything so far: the simplest context, working memory

Tool results become the next input

```
input = []
for block in output:                                     # output: show text, run tool requests
    if block.type == "text":
        print(block.text)
    if block.type == "tool_use":
        print(f"$ {block.input['cmd']}")
        done = subprocess.run(block.input["cmd"], shell=True, ...)
        print(done.stdout)
        input.append({..., "content": done.stdout or f"(exit {done.returncode})"})
```

(abridged: two long lines cut to `...`; the last one is commented "input: from the world")

No tool request means it's done

```
if input:
    messages.append({"role": "user", "content": input})
    continue
if not chat or (input := read("\n> ")) == "/q":
    break
messages.append({"role": "user", "content": input})
```

- `if input: ... continue` is the agent loop: results go in, call again
- No input from the world is termination: no step count, no magic word
- `chat` gives one loop two modes: run to completion, or hand back with `>`

Run it: two calls, one answer

```
uv run lessons/03-control-flow/quark.py "which quark.py in the lessons
folder is the longest? answer in one sentence"
```

```
$ find . -ipath '*lessons*' -name 'quark.py' -exec wc -l {} + 2>/dev/null | sort -n
  9 ./lessons/01-model-interface/quark.py
 33 ./lessons/02-input-and-output/quark.py
 46 ./lessons/03-control-flow/quark.py
234 ./lessons/04-context/quark.py
322 total
```

The longest is `./lessons/04-context/quark.py`, at 234 lines.

Control flow can be much more than one loop

- **Its shape:** single call, chat, workflow, agent loop, agents handing off, a loop woken by a message or a schedule
- **When it stops:** model stops asking, step count, spending limit, refusal, a person says stop
- **Who it hands back to:** a person, another program, or no one
- **What runs at once:** one model call, or several

[LangGraph](#) is a product built around this primitive alone.

control_flow.py stops, and says why

```
MAX_STEPS = 10
...
for step in range(1, MAX_STEPS + 1):
    ...
    if reply.stop_reason == "refusal":
        print("[stopped: the model declined]")
        break
    ...
    if not results:
        print(f"[done in {step} steps]")
        break
    messages.append({"role": "user", "content": results})
else:
    print(f"[stopped: hit the {MAX_STEPS}-step limit]")
```

Two runs: done, and out of steps

```
uv run lessons/03-control-flow/control_flow.py "which folder in lessons  
has the most files? answer in one English sentence"
```

```
The `02-input-and-output` folder in `lessons` has the most files, with 5.  
[done in 2 steps]
```

With `MAX_STEPS` set to 1, it stops before the model sees the result:

```
$ git --version  
git version 2.43.0  
  
[stopped: hit the 1-step limit]
```

(first run shortened)

Five workflows come up most

- **Prompt chaining:** a fixed sequence of calls
- **Routing:** one call sorts the input, your code picks the path
- **Parallelization:** calls at the same time, then combined
- **Orchestrator–workers:** one call splits the task, others do the pieces
- **Evaluator–optimizer:** one call produces, another judges, repeat

From Anthropic's [Building effective agents](#).

workflow.py: your code decides the order

- Draft, judge, rewrite, judge: no tools, no loop the model steers
- Stops when the judge replies `PASS` , or after three rounds
- One run (shortened):

```
--- draft 1 ---  
Agent rigs wrap model calls while tools guide small tasks.  
  
--- evaluator ---  
**Fails.** Two words are not five letters long:  
...  
--- draft 2 ---  
Agent frame holds model parts while tools guide small tasks.  
  
[evaluator: pass]
```

The rule: sequence the four, and decide when to stop

Arranged differently, the same four pieces are a single call, a chatbot, an agent, or a workflow.

What control flow never does

- Get the request to the model and the response back: **model interface**
- Gather what goes in: **input**
- Handle the response: **output**
- Decide what each request holds, even `messages`: **context**

Control flow only decides what runs, in what order, and when to stop.

What's missing: it's an agent that knows nothing

- Not where it is, what day it is, or who it is
- Not what you told it yesterday
- And `messages` grows every pass, until the model can't read it all

Something has to decide what the model sees.

Next: Context

Lesson 4